



Slide 1
CS 170
Java Programming 1
Multiple Choice
 Duration: 00:00:56
 Advance mode: Auto

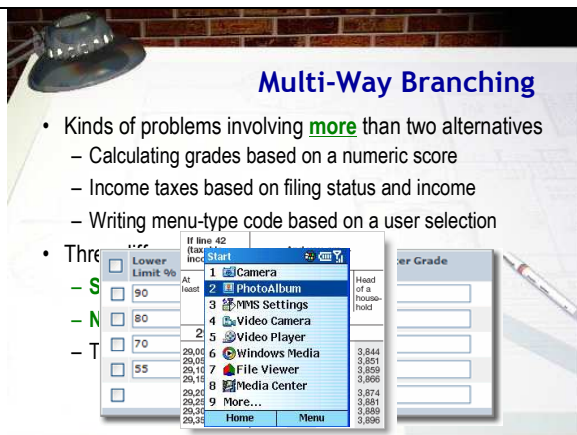
Notes:

Hi Folks. Welcome to the CS 170, Java Programming 1 lecture on Multi-Way Branching. It would be wonderful if every decision could be reduced to a single "either-or" choice that could be solved with a simple if-else statement; it would sure make programming easier!

Most of the time, though, things are more complex, and you'll need to write selection statements involving three, four, five or more alternatives. This is called multi-way branching.

In this lecture, you'll learn how to create multi-way branches using sequential and nested if statements. You'll also learn how to use JUnit to hand-code the unit tests to ensure that your programs involving multi-way branches are completely tested, a technique called branch coverage testing.

Let's look briefly at different situations that require more than one alternative.



Slide 2
Multi-Way Branching
 Duration: 00:01:54
 Advance mode: Auto

Notes:

Here are three different examples of situations that can't be handled with a simple if or even with and if else:

- When calculating grades based on a numeric score, like your grade in this class, you have one numeric input, but a different output, and thus a different branch, for each possible grade.
- If you want to write a program to calculate your income tax using the tax tables, you'll have at least two inputs—your filing status, married, single and so on, and your income. When you put those two together, as you can see from the portion of the tax table shown here, you end up dealing with literally hundreds of different alternatives.
- Finally, there's your every-day drop-down menu, like this, where the user chooses 1, 2 or 3 and a particular program is launched or an action is taken.

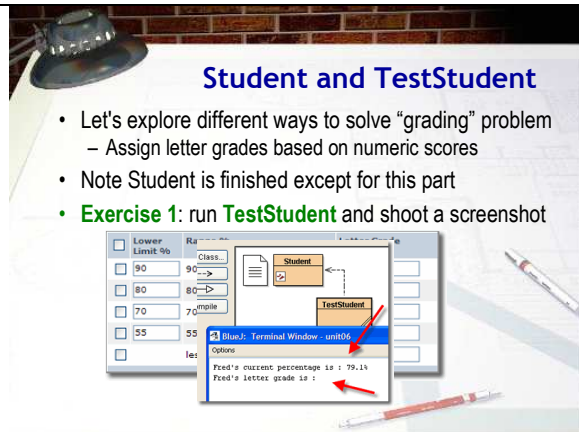
To solve these kinds of problems you'll need to learn how to use one of Java's multi-way branching statements. Java has specific techniques that are designed for each of these kinds of situations.

When you have related comparisons, that is, where one test condition can affect another, like in our grading example, you'll use sequential if statements. In cases where your decisions are leveled, that is, they involve different kinds of inputs such as filing status and income, you'll use nested if statements. Finally, when you have independent blocks of code that can be numbered, as in the pop-up menu example, you can use Java's switch statement.

In this lesson, we'll look at using sequential if-else-if statements and using nested if statements. We'll look at the

switch statement in a later lesson.

For the problems this week, I've prepared a BlueJ starter project which you can download from the class Web site. Open the project and we'll start working through some of the examples.



Student and TestStudent

- Let's explore different ways to solve "grading" problem
 - Assign letter grades based on numeric scores
- Note Student is finished except for this part
- Exercise 1:** run **TestStudent** and shoot a screenshot

The screenshot shows the BlueJ IDE with a 'Student' class and a 'TestStudent' class. The 'TestStudent' class has a 'main' method. The output window shows: 'Fred's current percentage is : 79.1%' and 'Fred's letter grade is : '.

Slide 3

Student and TestStudent

Duration: 00:01:27

Advance mode: Auto

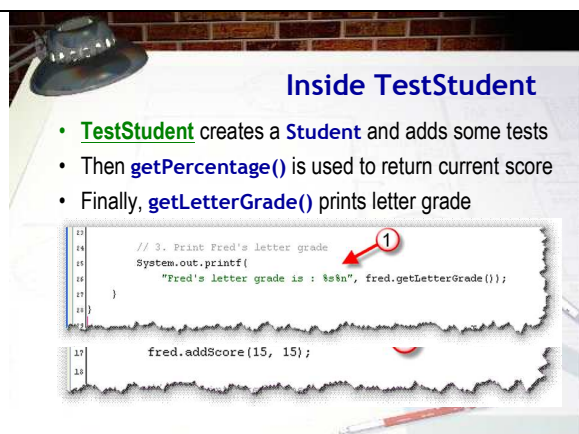
Notes:

One sequential comparison that you're all familiar with is the "letter grading scale" used to assign marks in school, (including in this course), similar to that shown in this table. Typically, your letter grade is based on a percentage representing the weighted average for all of the work you've done during the term. In the example shown here, for instance, those who earn 90% or greater receive 'A's, those with percentages less than 90%, but at least 80% earn 'B's, and so on.

To explore sequential comparisons let's write some code that uses selection to perform this calculation. The programs in the starter project, TestStudent.java and Student.java contain a Student class and a test program. Go ahead and run the program by right-clicking TestStudent.java and choosing Run Main Method from the popup menu.

The test program creates a Student object, adds a couple of scores, and the prints out the student's current grade. As you can see here, the program correctly calculates Fred's percentage score (79.1%), but doesn't yet handle the letter grade. That's our next stop.

Before we get there, though, let's take a look at what we've got; we'll read through both programs together, and then look at two different ways we can implement the code for calculating the letter grade.



Inside TestStudent

- TestStudent** creates a **Student** and adds some tests
- Then **getPercentage()** is used to return current score
- Finally, **getLetterGrade()** prints letter grade

The screenshot shows the TestStudent.java code. Line 14 creates a Student variable named fred. Lines 15-17 add three scores to the object. Line 20 sends the getPercentage() message to fred. Line 21 prints the results in percentage format. Line 22 prints the letter grade. A red arrow points to line 22.

Slide 4

Inside TestStudent

Duration: 00:01:52

Notes:

Here's how the test program works:

1. Line 14 creates a Student variable named fred, passing the String "Fred" to the constructor. Lines 15-17 add three scores to the object: on the first exam, Fred got a 75 out of 100, on the second, he got an 80 out of 100, and on the third, a "pop" quiz, he got a perfect score—15 out of 15:

2. On line 20, the getPercentage() message is sent to fred, and the results stored in a double variable named percent. Then, on line 21 and 22, the printf() method is used to print the results in percentage format. Note the three different parts of the format specifier that's used:

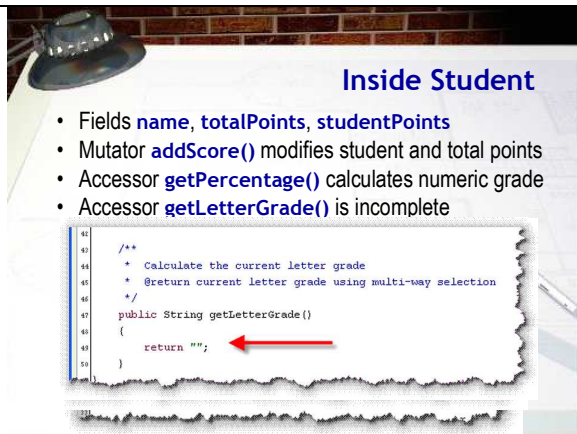
- `%.1f` says that the second parameter will be a floating point number and we'll display it showing one place to the

Advance mode: Auto

right of the decimal. No field width is specified.

- %% following say "insert a literal percent symbol." A single percent sign would be interpreted as a placeholder character.
- the final %n says to insert a newline character in the output. Using %n is better than using \n, since it will be replaced with the correct line-feed character for the platform that your program is running on.

3. Finally, on line 25, Fred's current letter grade is printed by calling using printf() once again, this time with the %s specifier to display String output, and the getLetterGrade() method is called to calculate the letter grade.



Inside Student

- Fields **name**, **totalPoints**, **studentPoints**
- Mutator **addScore()** modifies student and total points
- Accessor **getPercentage()** calculates numeric grade
- Accessor **getLetterGrade()** is incomplete

```
42  /**
43   * Calculate the current letter grade
44   * @return current letter grade using multi-way selection
45   */
46  public String getLetterGrade()
47  {
48      return "";
49  }
```

Slide 5

Inside Student

Duration: 00:01:58

Advance mode: Auto

Notes:

Now, let's take a look at the Student class:

1. The Student class has three instance variables, defined on lines 12-14. The totalPoints field will contain the total possible points for all exams and assignments that the student has completed, while the studentPoints field will contain the sum of the student's scores/

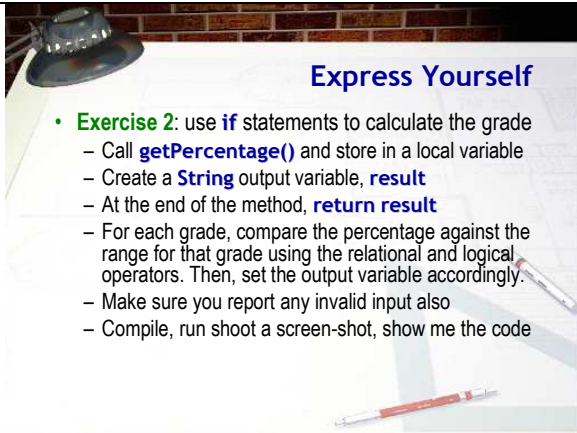
The single constructor initializes the name field.

2. When a Student object is first created, both the totalPoints fields and the studentPoints fields are set to 0. To change this, you use the mutator method addScore(). The addScore() method takes two arguments: the number of points the student earned on the assignment, and the total points possible on the assignment. It then adds each of these to the appropriate field.

3. The getPercentage() method returns the student's current numeric grade, as a percentage as the name implies. Note that the method multiplies studentPoints by 100 before dividing by totalPoints so that 75% will be returned as 75.0, not .75. Because the calculation uses floating-point division, not integer division, calling getPercentage() before entering a quiz will return the special value NaN which stands for not-a-number. If we had used integer division, we'd have to use an if statement if totalPoints was 0, and then decide what kind of error value to return.

4. The final Student method is getLetterGrade() which returns the letter grade based on the student's current percentage score. As you can see here, the method currently returns the empty string.

Now see if you can implement the getLetterGrade() method.



Express Yourself

- **Exercise 2:** use **if** statements to calculate the grade
 - Call **getPercentage()** and store in a local variable
 - Create a **String** output variable, **result**
 - At the end of the method, **return result**
 - For each grade, compare the percentage against the range for that grade using the relational and logical operators. Then, set the output variable accordingly.
 - Make sure you report any invalid input also
 - Compile, run shoot a screen-shot, show me the code

Slide 6 

Express Yourself

Duration: 00:01:53

Advance mode: Auto

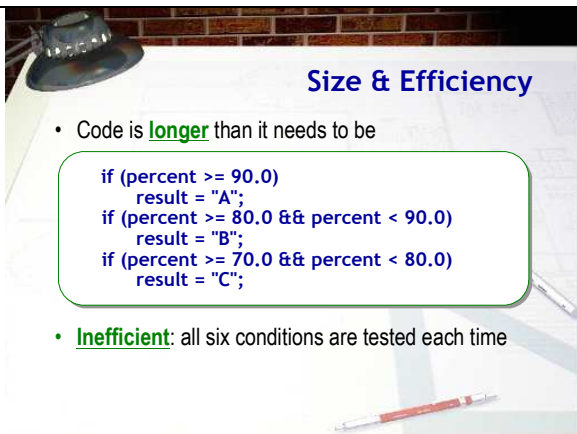
Notes:

Since you already know how to use the if statement and the logical operators let's see if you can use those to solve this problem. Here's what you should do:

- Call the `getPercentage()` method to retrieve the student's current numeric grade, and store the result in a double local variable named `percent`. You could eliminate this variable, but then you'd have to call the `getPercentage()` method each time you wanted to test the current percentage grade, making the program much more inefficient.
- Create an output variable, a `String` named `result`, and initialized it to the empty string.
- At the end of the method, add a return expression that just returns your variable `result`.
- Between the points where you initialize and then return `result`, add code to compare the current numeric percentage each range for each possible letter grade. Use the `&&` operator as part of each if statement to check the lower and upper bound for the range. If you get a match, then set the `result` variable appropriately.
- Make sure that you handle percentages less than 0 and percentages > 100 by returning an "invalid input" error message. Don't print your message, just have `getLetterGrade()` return it instead of a grade.

When you're done, compile and run the test program and make sure that it works. Then, shoot me a screen-shot of the output. Then, copy and paste your code below the screen-shot.

Now, let's take a look at the pitfalls and dangers of using independent if statements like this.



Size & Efficiency

- Code is **longer** than it needs to be

```
if (percent >= 90.0)
    result = "A";
if (percent >= 80.0 && percent < 90.0)
    result = "B";
if (percent >= 70.0 && percent < 80.0)
    result = "C";
```

- **Inefficient:** all six conditions are tested each time

Slide 7 

Size & Efficiency

Duration: 00:01:13

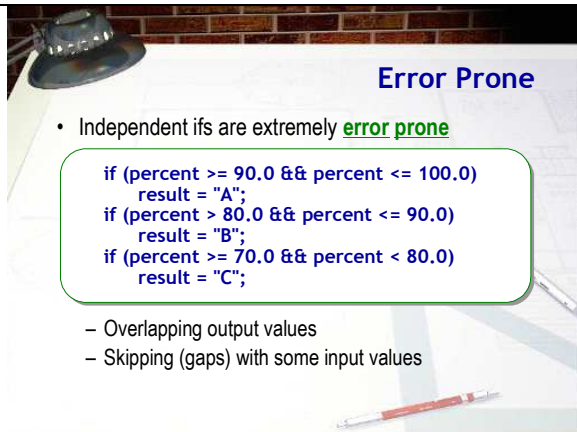
Advance mode: Auto

Notes:

While your solution is correct, it has several drawbacks:

- First, it's longer than it needs to be. Because each if statement is independent, you have to supply both an upper and a lower bounds. You can't just replace the second condition here with `if (percent >= 80.0)` because that would reset all of the 'A' grades to 'B' grades, because all of the percentages that are ≥ 90.0 are also ≥ 80.0 . You still need to check that the percent is less than 90 even though the previous statement checked to see if it was greater than 90. If the previous ($\geq 90\%$) condition was false, then < 90.0 has to be true, but we still have to type it in.
- Second, it's quite inefficient. Suppose, that a student got a 95 for percent and you set `result` to "A". You still have to evaluate the remaining six Boolean conditions, even though they couldn't possibly be true. With independent if

statements, every if condition is tested, even after the correct one has been found.



Error Prone

- Independent ifs are extremely **error prone**

```
if (percent >= 90.0 && percent <= 100.0)
    result = "A";
if (percent > 80.0 && percent <= 90.0)
    result = "B";
if (percent >= 70.0 && percent < 80.0)
    result = "C";
```

- Overlapping output values
- Skipping (gaps) with some input values

Slide 8

Error Prone

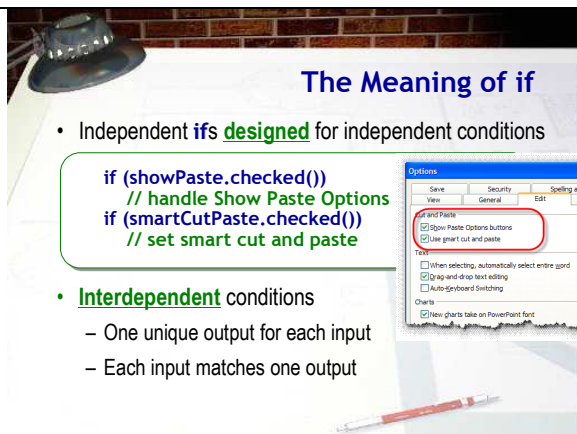
Duration: 00:01:01

Advance mode: Auto

Notes:

Independent ifs also tend to trick you into making two very common errors. Here's some code that's very similar to that on the previous page, except this code isn't really correct.

- First, note that some input values actually map to two output values. If a student gets exactly 90% for percent, both the first if condition and the second if condition will be true. The "B" will end up overwriting the (correct) "A" because the second relational test in the second condition should be `percent < 90.0` not `percent <= 90.0`.
- Second, note that some input values aren't mapped to a letter grade at all. This is also very easy to do. In this case if a student gets exactly 80%, then there is no result assigned.



The Meaning of if

- Independent ifs **designed** for independent conditions

```
if (showPaste.checked())
    // handle Show Paste Options
if (smartCutPaste.checked())
    // set smart cut and paste
```

- Interdependent** conditions
 - One unique output for each input
 - Each input matches one output

Slide 9

The Meaning of if

Duration: 00:01:39

Advance mode: Auto

Notes:

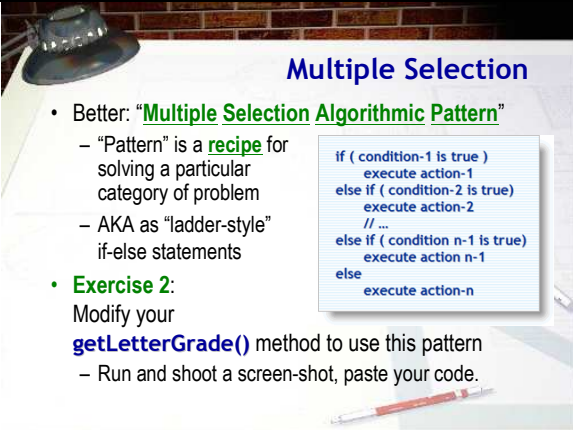
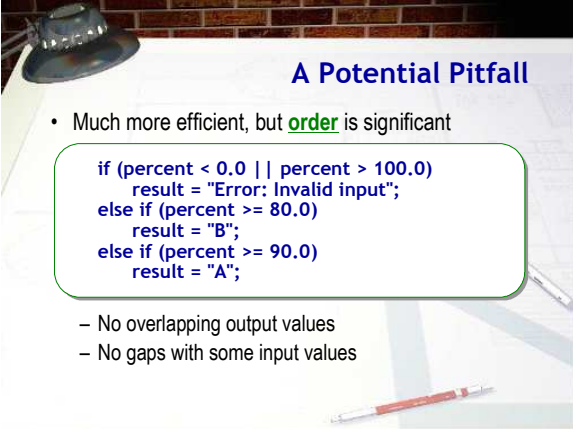
The real problem with using independent ifs in this situation is that the feature isn't really a good match for the solution that you've designed. Independent if statements were designed when you have entirely independent conditions and each condition controls one output.

Here's an example. Suppose you had an options dialog like this one from PowerPoint, with several different checkboxes that controlled different aspects of your program's operation. People using the dialog should be able to check all of the options, none of the options, or any combination in between. Each option is completely independent—none of the tests and none of the actions affect any other action.

This is what independent ifs were designed for, and that's really the only time you should use independent if statements. Semantically, that's what independent if statements really mean.

So, let's look back at our problem. Notice that each of the if statements really depends upon the other (or should, anyway). We call these interdependent conditions, where a related series of if statements work together to produce one single output.

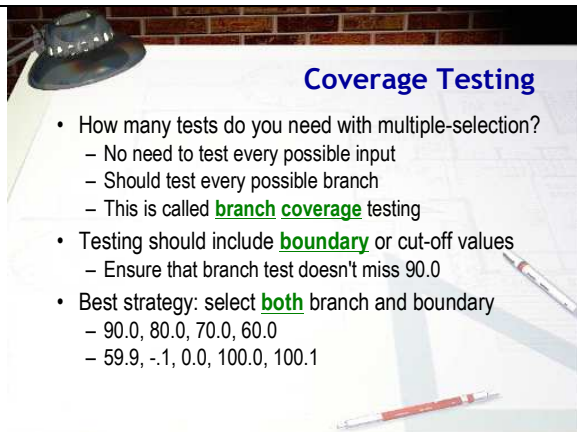
- Given a particular input, only one of the series of

	conditions should be true. In other words we don't want a situation where the same percentage actually evaluates to two different grades. The series of conditions should cover every possible input. We don't want an input value to pass through unselected.
 Slide 10 🎧 Multiple Selection Duration: 00:01:19 Advance mode: Auto	Notes: A better solution, when you need to select one course of action from many possible alternatives (which is the case here), is to employ what Rick Mercer at Arizona State calls the "Multiple Selection Algorithmic Pattern", shown here. (An "algorithmic pattern" is simply a recipe for solving a particular category of problem.) This particular pattern is also often called a ladder style if statement, because each of the conditions are formatted one under the other, like the rungs on a ladder. It is also sometimes called an if-else-if statement. (Note though that the else if keywords are not combined into a single elseif, like they are in some other languages, such as Visual Basic.) OK, now it's your turn again. Go ahead and modify your getLetterGrade() method to use this new pattern. Rather than erasing all of the code you've written, just surround it with a set of multi-line comment characters (this is called commenting-out a section of code), then make a copy and modify the copy. When you're done, run the test program and shoot me a screen-shot. Then, copy and paste the new code that you've written.
 Slide 11 🎧 A Potential Pitfall Duration: 00:01:22 Advance mode: Auto	Notes: While the multiple-selection pattern is much more efficient than the independent if statements, each of the statements is dependent on the one that comes before it. That means that the order of the statements is significant. If you switch the order of the "A" and "B" tests, for instance, like this: <pre> else if (percent >= .80) result = "B"; else if (percent >= .90) result = "A"; </pre> all of those students who should have gotten "A"s, will get "B"s instead, since a score of .95 matches the condition (percent >= .80) and that condition is now encountered before the condition that tests for an "A". The test for an "A" will never be

executed.

You can still make mistakes assigning the wrong grade if you use the wrong relational operator. If you were to use percent > 90 for one condition, then students with a 90 would get a B, not an A. There are no overlapping output values, though. Every input will map to exactly one action. You also can't accidentally create a gap so that a certain value isn't processed. The Multiple-selection pattern ensures that every input will be given a corresponding output value.

So, how do we fix this one last problem, where a student is assigned a wrong grade? By a specific kind of testing called coverage testing.



Coverage Testing

- How many tests do you need with multiple-selection?
 - No need to test every possible input
 - Should test every possible branch
 - This is called **branch coverage** testing
- Testing should include **boundary** or cut-off values
 - Ensure that branch test doesn't miss 90.0
- Best strategy: select **both** branch and boundary
 - 90.0, 80.0, 70.0, 60.0
 - 59.9, -.1, 0.0, 100.0, 100.1

Slide 12 
Coverage Testing
Duration: 00:02:31
Advance mode: Auto

Notes:

You many tests would you need to write to test the `getLetterGrade()` method? Well, to make sure that every possible value is handled correctly, you'd need to call the method with all of the numbers between -1.0 through 101.0. Because the method parameter is a real number, though, that means you'd have an infinite number of tests: 1.0, 1.1, 1.11, 1.111 and so on. If we used integers for the percentages, testing every possible value would be possible, but it really isn't necessary.

When you use multiple-selection, what you need is to make sure that every possible branch through the code is executed. This is called coverage testing. With coverage testing, you make sure that every line of code is executed: with if...else statements that means you have to test both the true and false branches.

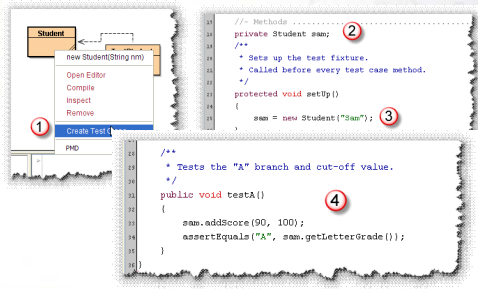
Your testing should also include the cut-off or boundary values. This would help you avoid situations where students with 90. are accidentally given the grade "B" instead of "A" because the boolean condition was accidentally written as percent > 90 instead of percent >= 90. If you don't include boundary conditions, you might test both the "A" and "B" branches and miss this case.

The best (or at least shortest) testing strategy is to select test values that combine both boundary and branch conditions, such as the value 90.0. This checks the path for returning an "A" and also, at the same time tests the boundary or cutoff condition for an "A". Counting down by tens to 60 checks all of the boundaries, but still leaves three paths un-tested: the "F" path which you can check with 59.9, the less-than-zero out-of-bounds path, which you can check with -.1 and the greater than 100 out-of-bounds path which you can check with 100.1. You'll also need the in-bounds values 0.0 and 100.0 as well. So, to completely test the method, we'll need nine different tests.

Now that we have the test data, let's look at using BlueJ to write JUnit unit tests manually.

Hand-coding JUnit Tests

- **Exercise 3:** let's hand-code some JUnit unit tests



Slide 13

Hand-coding JUnit Tests

Duration: 00:02:25

Advance mode: Auto

Notes:

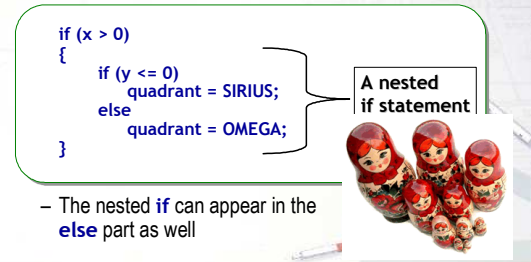
The TestStudent program really isn't up to testing each of these branches, so we'll turn to JUnit. While BlueJ's automatic unit-testing recording is useful when you're first learning how to write unit tests, it can be pretty cumbersome when you have to write a lot of them, like you do when writing branch-coverage unit tests. In this case, I find hand-coding the JUnit tests is actually a lot easier. Here are the steps to follow:

1. Right-click the Student class and select Create Test Class from the context menu. This will create the JUnit class StudentTest. Double-click StudentTest to open it in an editor and then add your name and a file comment to the top of the file.
2. Create an instance variable of the class we're testing. I've called my Student variable sam.
3. Locate the setUp() method. Inside setUp(), initialize your variable as I've done here in the screen-shot. This is the test-fixture that you created using the menu options previously.
4. For each test you want to do, you create a void method that begins with the word "test". Here is the testABranch() method. Notice that the method first modifies the Student object, by adding a single quiz score of 90 out of 100. You'll have to modify the Student object in every test method, since the setUp method is called right before your test method is run, so the Student variable is always freshly initialized. The second line in the test method uses the JUnit method assertEquals which takes two parameters: the expected value and the actual value. For the actual value, call your function like this.

For the each of the remaining branch tests, just copy the method shown here and change the values passed to addScore() to generate the range of input numbers you want to test. When you're done, run the JUnit tests and shoot me a screen-shot of all the tests running green. Then, copy and paste your test code into your IC exercise document.

Nested if Statements

- Can also combine Boolean expressions with **nesting**
 - One **if** statement embedded inside the body of another



Slide 14

Notes:

As you saw in the last section, you can code decisions involving multiple alternatives in two different ways:

- by writing independent if statements that combine their different boolean conditions using the AND and the OR operators.
- by arranging your if statements in a ladder-like sequence of related, dependent if statements.

A third way to do the same thing is through nesting.

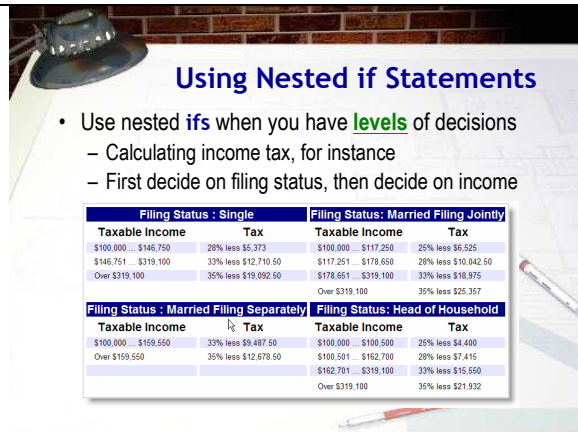
Nesting means that one if statement is embedded or nested inside the body of another if statement, much like the traditional Russian nesting dolls, shown here. The nested if

Nested if Statements

Duration: 00:00:48

Advance mode: Auto

statement can appear in the body of the if portion of an if-else statement, in the body of the else portion, or inside both.



- Use nested **ifs** when you have **levels** of decisions
 - Calculating income tax, for instance
 - First decide on filing status, then decide on income

Filing Status: Single		Filing Status: Married Filing Jointly	
Taxable Income	Tax	Taxable Income	Tax
\$100,000 ... \$146,750	28% less \$5,373	\$100,000 ... \$117,250	25% less \$6,525
\$146,751 ... \$319,100	33% less \$12,710.50	\$117,251 ... \$178,650	28% less \$10,042.50
Over \$319,100	35% less \$19,092.50	\$178,651 ... \$319,100	33% less \$18,975

Filing Status: Married Filing Separately		Filing Status: Head of Household	
Taxable Income	Tax	Taxable Income	Tax
\$100,000 ... \$159,550	33% less \$9,487.50	\$100,000 ... \$100,500	25% less \$4,400
Over \$159,550	35% less \$12,678.50	\$100,501 ... \$162,700	28% less \$7,415
		\$162,701 ... \$319,100	33% less \$16,550
		Over \$319,100	35% less \$21,932

Slide 15

Using Nested if Statements

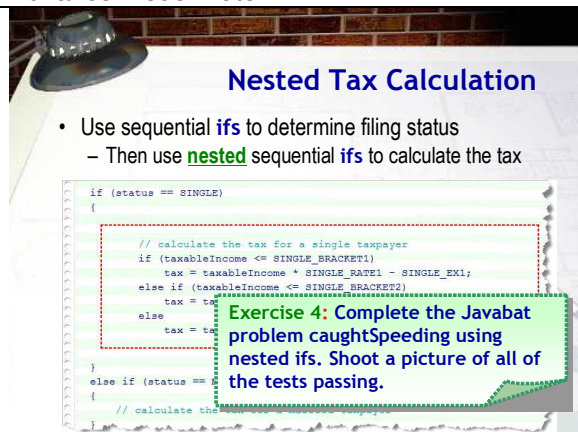
Duration: 00:00:52

Advance mode: Auto

Notes:

Nested if statements are used whenever you need to test for different levels of decision making, rather than a set of purely sequential tests. If you're one of those fortunate folks making more than a hundred thousand dollars a year, for instance, you'll need to calculate your taxes using the following formula, instead of using the tax tables.

Suppose that you were single, with a taxable income of \$159,750. To calculate your tax, you'd first locate the schedule for your filing status (Single), and then locate the bracket you fall into (the second). Your tax would be \$159,750 x 33% less \$12,710.50.



- Use sequential **ifs** to determine filing status
 - Then use **nested** sequential **ifs** to calculate the tax

```
if (status == SINGLE)
{
    // calculate the tax for a single taxpayer
    if (taxableIncome <= SINGLE_BRACKET1)
        tax = taxableIncome * SINGLE_RATE1 - SINGLE_EX1;
    else if (taxableIncome <= SINGLE_BRACKET2)
        tax = tax;
    else
        tax = tax;
}
else if (status == ...
{
    // calculate the
```

Exercise 4: Complete the Javabat problem caughtSpeeding using nested ifs. Shoot a picture of all of the tests passing.

Slide 16

Nested Tax Calculation

Duration: 00:01:48

Advance mode: Auto

Notes:

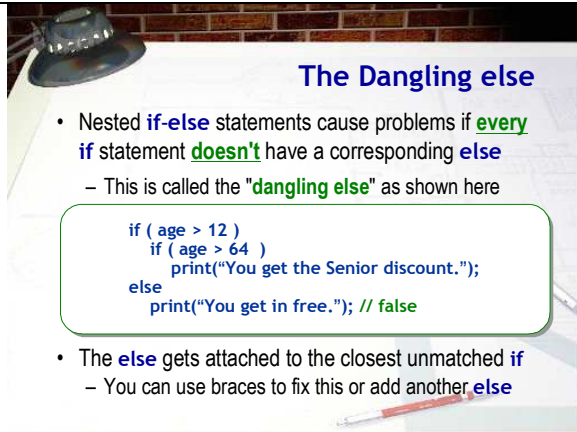
If you wanted to write a program to perform this calculation, you'd need to follow a similar set of steps. You'd first need to use a set of sequential if statements to determine which set of calculations to use, like this. Then, nested inside the body of each portion of each if statement, you'd test the various income levels, like this.

Let's try an exercise with nested if statements on Javabat, since the getLetterGrade() problem only has one input. We'll come back to getLetterGrade() in the next lecture. On the Javabat Web site, locate the caughtSpeeding problem in the logic section. Here's what the problem says:

You are driving a little too fast, and a police officer stops you. Write code to compute the result, encoded as an int: no ticket=0, small ticket=1, big ticket=2. If speed is 60 or less, the result is 0. If speed is between 61 and 80 inclusive, the result is 1. If speed is 81 or more, the result is 2. Unless it is your birthday -- on that day, your speed can be 5 higher in all cases.

Notice that the method has two kinds of input—your speed and whether it is your birthday or not, just like the income-tax problem has two kinds of input—your filing status and your income.

Once you've solved the problem, shoot me a screen-shot showing your solution with all of the tests passing.



The Dangling else

- Nested **if-else** statements cause problems if **every** **if** statement **doesn't** have a corresponding **else**
 - This is called the "**dangling else**" as shown here

```
if ( age > 12 )
  if ( age > 64 )
    print("You get the Senior discount.");
  else
    print("You get in free."); // false
```

- The **else** gets attached to the closest unmatched **if**
 - You can use braces to fix this or add another **else**

Slide 17 🎧
The Dangling else
Duration: 00:01:18
Advance mode: Auto

Notes:

When you nest one if statement inside of another, the nested if statement won't always have an else portion. When this occurs, it's possible for the else clause of the enclosing block to inadvertently "attach itself" to the inner if statement. This is known as the **dangling else** problem.

Here's an example. In the code shown in the screenshot, the programmer intended to allow children 12 and under to "get in free" (the else branch of the first if statement. A nested if statement was intended to provide a "senior citizens" discount as well. Everyone else was supposed to pay full price.

When the code is run, though, the result is exactly the opposite. Because the else clause automatically attaches itself to the closest unmatched if statement, the young kids all pay full price, while all of the pre-senior adults get in for free.

Avoiding this problem is really pretty easy: whenever you use nested if statements, always enclose the body of both the if and else portions in braces. The braces act like a "fence" that keeps the else portions where they belong.